

Contents

1	Part 1	2
1.1	Task 1.1 Determine the analytical marginal prior mean and variance of a Gaussian process, $f(x) \sim GP(0, k_i(x, x'))$, i.e. compute $\mathbb{E}[f_i(x)]$ and $\mathbb{V}[f_i(x)]$ for each of the six covariance functions (for $i = 1, 2, \dots, 6$).	2
1.2	Task 1.2 Which of the six covariance functions are stationary covariance functions?	2
1.3	Task 1.3 Let $X = x_i^1 00_{i=1}$ be a sorted set of equidistant points in the interval $[0, 2]$. Figure 1 shows the covariance matrices for function values evaluated at X as well as samples from the corresponding Gaussian process prior for each of the six covariance functions. Match the plots to each of the six covariance functions.	3
1.4	Task 1.3 Implement the kernel function k_7 . Generate and plot $S = 30$ realizations (samples) of the process $f(x) \sim GP(0, k_7(x, x'))$ for $x \in [-6, 6]$ for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 0, \frac{1}{2})$. Repeat with $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 0, 1, \frac{1}{2})$ and $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 1, \frac{1}{2})$.	3
2	Part 2	6
2.1	Task 2.1 Generate $S = 100$ samples from the prior $p(w)$ and plot the corresponding functions $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.	6
2.2	Task 2.2 What prevents us from using the equations in section 3.3 in Murphy1 to compute the exact posterior distribution of the parameters given the data analytically for the system in eq. (14) ?	7
2.3	Task 2.3 Implement a python for function for evaluating the logarithm of the joint density in eq. (14). Report the numerical value of $\log p(y, w)$ for the dataset given above when $w_0 = w_1 = 0$	7
2.4	Task 2.4 Determine the expression for the gradient and Hessian of the logarithm of the joint density wrt. w	8
2.5	Task 2.5 Determine the Laplace approximation $q(w)$ of the posterior distribution such that $p(w y) \approx q(w)$	9
2.6	Task 2.6 Create a 2D contour plot of posterior distribution of the parameters. Plot the contours of the Laplace approximation on top of the plot to validate your results from the previous tasks.	10
2.7	Task 2.7 Generate and plot $S = 100$ samples from the approximate posterior distribution. Pot the corresponding function $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.	11
2.8	Task 2.8 Use the Laplace approximation to compute the (approximate) posterior probability for the event $f(8) > 3$	12
2.9	Task 2.9 Determine the mean and a 95% interval for the (approximate) distribution $p(y^* y, x^*)$ for $x^* = 3.75$	13

1 Part 1

1.1 Task 1.1 | Determine the analytical marginal prior mean and variance of a Gaussian process, $f(x) \sim GP(0, k_i(x, x'))$, i.e. compute $\mathbb{E}[f_i(x)]$ and $\mathbb{V}[f_i(x)]$ for each of the six covariance functions (for $i = 1, 2, \dots, 6$).

We are given six covariance functions, and we know that:

$$m(x) = \mathbb{E}[f(x)] \quad (1.1)$$

Where:

$$f(x) \sim GP(m(x), k(x, x')) \quad (1.2)$$

Thus for all six covariance functions, we can see from the task description that $m(x) = 0$, thus:

$$m(x) = \mathbb{E}[f(x)] = 0 \quad (1.3)$$

Furthermore, since the mean prior mean is 0, it is then given that the prior variance is:

$$\mathbb{V}[f(x)] = k(x, x) \quad (1.4)$$

Thus we have:

$$k_1(x, x) = 2 \exp(0) \quad (1.5)$$

$$k_2(x, x) = \exp(0) \quad (1.6)$$

$$k_3(x, x) = 4 + 2x^2 \quad (1.7)$$

$$k_4(x, x) = \exp(-2 \sin(0)^2) \quad (1.8)$$

$$k_5(x, x) = \exp(-2 \sin(0)^2) + 4x^2 \quad (1.9)$$

$$k_6(x, x) = \frac{1}{5} \quad (1.10)$$

We have hereby determined the prior mean to be 0 and found each of the variances given as $k_i(x, x)$.

1.2 Task 1.2 | Which of the six covariance functions are stationary covariance functions?

From the weekly exercises, we learned that stationary depends on the difference between two inputs (shift invariant) and depends on $x - x'$, meanwhile isotropic means rotation invariant, so $\|x - x'\|$.

We see that functions 1, 2 and 4 depends on $(x - x')$, meanwhile 3 and 5 has a term xx' , thus not stationary, and 6 has $\min(x, x')$ so it is not stationary either.

Therefore, covariance functions 1, 2, and 4 are stationary.

1.3 Task 1.3 | Let $X = x_i^1_{00_{i=1}}$ be a sorted set of equidistant points in the interval $[0, 2]$. Figure 1 shows the covariance matrices for function values evaluated at X as well as samples from the corresponding Gaussian process prior for each of the six covariance functions. Match the plots to each of the six covariance functions.

First off, we can recognize that (a) and (d) are from exponential kernels, and (a) is specifically from function 2 as the κ value is 1 which matches the maximum value of the kernel, and (d) is function 1 with a maximum value of 2.

Next, plot (b) is from function 6, as the maximum value of the kernel is 2.2 which matches with $\frac{1}{5} + 2 = 2.2$.

Furthermore, plot (f) is from function 3, since the maximum value is 12 which matches with $4 + 2 \cdot 2 \cdot 2 = 12$.

Finally, (c) belongs to function 5 (it produces higher values due to $4xx'$), and (e) belongs to function 4. To also further elaborate, we see that plot (c) has higher variance which corresponds to the non-stationary term $4xx'$, and higher variance is exactly non-stationary behavior, so this confirms that (c) belongs to function 5.

1.4 Task 1.3 | Implement the kernel function k_7 . Generate and plot $S = 30$ realizations (samples) of the process

$f(x) \sim GP(0, k_7(x, x'))$ for $x \in [-6, 6]$ for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 0, \frac{1}{2})$.
Repeat with $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 0, 1, \frac{1}{2})$ and
 $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 1, \frac{1}{2})$

First this is the code that was written, where we had to change the hyperparamter class and create a new kernel 7 function as well as adapt already existing code:

```

1 @dataclass
2 class Hyperparameters(object):
3     kappa0: float = 1.0
4     kappa1: float = 1.0
5     kappa2: float = 1.0
6     lengthscale: float = 1.0 # characteristic lengthscale, positive scalar (
7
8     def to_array(self):
9         """ return hyperparameters as flat JaX-array (to be used later) """
10        return jnp.array([self.kappa0, self.kappa1, self.kappa2, self.lengthscale])
11    @staticmethod
12    def from_array(hyper_array):
13        """ instantiates Hyperparameter object from flat JaX-array (or list) of hyperparameters
14            (to be used later) """
15        kappa0, kappa1, kappa2, lengthscale = hyper_array
16        return Hyperparameters(kappa0, kappa1, kappa2, lengthscale)
17
18    def __repr__(self):
19        """ for reporting hyperparameter values """
20        return f'Hyperparameters(kappa0={self.kappa0:3.2f}, kappa1={self.kappa1:3.2f}, kappa2={self.kappa2:3.2f}, lengthscale={self.lengthscale:3.2f})'

```

```

1 def k7(X1, X2, hyper):
2     # X1: NxD, X2: MxD
3     x1 = X1[:, None, :] # (N,1,D)
4     x2 = X2[None, :, :] # (1,M,D)
5
6     # linear term
7     linear = hyper.kappa1**2 * (x1 * x2).sum(axis=-1)
8
9     # squared exponential term
10    sqdist = jnp.sum((x1 - x2)**2, axis=-1)
11    se = hyper.kappa2**2 * jnp.exp(-sqdist / (2 * hyper.lengthscale**2))
12
13    # constant term
14    const = hyper.kappa0**2
15
16    return const + linear + se

```

```

1 # create an Nx1 vector of equidistant points in [-6, 6]
2 N = 30
3 X = jnp.linspace(-6, 6, N)[: , None]
4
5 # number of samples to be plotted
6 num_samples = 10
7
8 # specify random seed
9 key = random.PRNGKey(1)
10
11 # specify hyperparameters
12 hyper = Hyperparameters(
13     kappa0=5.0,
14     kappa1=2.0,
15     kappa2=0.0,
16     lengthscale=0.5
17 )
18
19 # instantiate kernel object and construct kernel
20 K = k7(X, X, hyper)
21
22 # plot
23 fig, ax = plt.subplots(1, 2, figsize=(20, 5))
24
25 im = ax[0].pcolormesh(X.flatten(), X.flatten(), K, shading='auto')
26 ax[0].set(xlabel='Input $x$', ylabel='Input $x$', title=f"Kernel function for $\backslash\backslash\kappa_0$ = {
27     hyper.kappa0:2.1f}, $\backslash\backslash\kappa_1$ = {hyper.kappa1:2.1f}, $\backslash\backslash\kappa_2$ = {hyper.kappa2:2.1f}
28     and $\backslash\backslash\ell$ = {hyper.lengthscale:2.1f}")
29 ax[0].grid(False)
30 ax[0].set_aspect('equal')
31 add_colorbar(im, fig, ax[0])
32
33 m = jnp.zeros(len(X))
34 f_samples = generate_samples(key, m, K, num_samples=num_samples, jitter=1e-8)
35 ax[1].plot(X, f_samples, alpha=0.75, linewidth=3);
36 ax[1].grid(True)
37 ax[1].set(xlabel='$x$', ylabel='$f(x)$', title='Samples from the Gaussian process');

```

Then we just try with the different sets of hyperparameters.

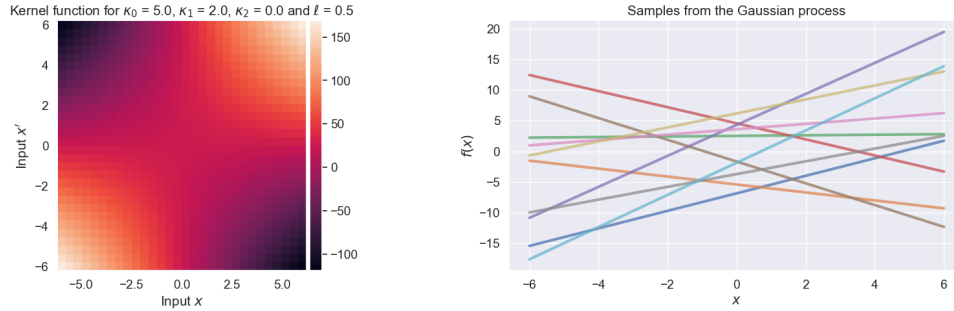


Figure 1.1: Covariance matrices and samples from the corresponding Gaussian process prior distribution for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 0, \frac{1}{2})$

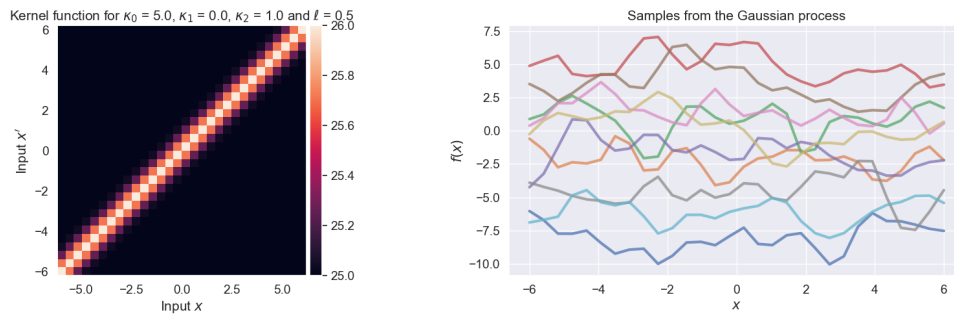


Figure 1.2: Covariance matrices and samples from the corresponding Gaussian process prior distribution for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 0, 1, \frac{1}{2})$

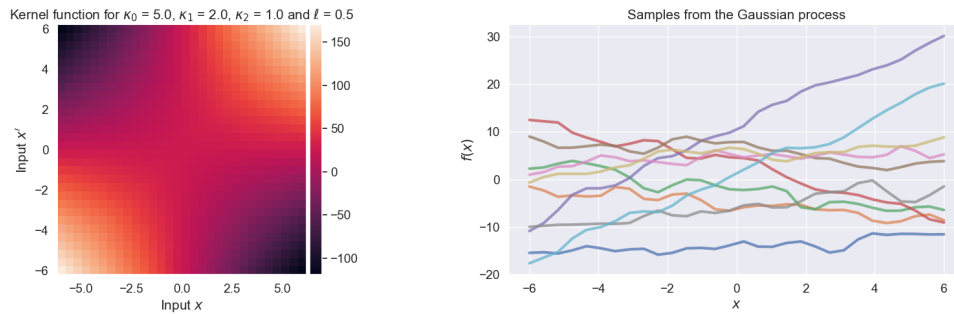


Figure 1.3: Covariance matrices and samples from the corresponding Gaussian process prior distribution for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 1, \frac{1}{2})$

We have, thus, implemented kernel function k_7 and generated a plot for $S = 30$ realizations for the three different sets of hyperparameters.

2 Part 2

2.1 Task 2.1 | Generate $S = 100$ samples from the prior $p(w)$ and plot the corresponding functions $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.

We write the following code shown below:

```
1 tau = 2
2
3 x = np.array([9.589,7.375,4.647,2.501,2.538,6.783,4.294,5.111,0.130, 0.783])
4 y = np.array([3.032,3.349,2.906,2.126,1.538,2.787,3.078,2.993,0.828,-0.331])
5
6 def sigma(x):
7     return np.tanh(x)
8
9 def f(x, w):
10    return w[1] * sigma(x + w[0])
11
12 x_plot = np.linspace(0, 10, 100)
13
14 cov = tau**2 * np.eye(2)
15
16 for _ in range(5):
17     w = np.random.multivariate_normal(mean=[0, 0], cov=cov)
18     plt.plot(x_plot, f(x_plot, w), alpha=0.7)
19
20 plt.scatter(x, y, color="black", label="Data")
21 plt.legend()
22 plt.show()
```

Which plots:

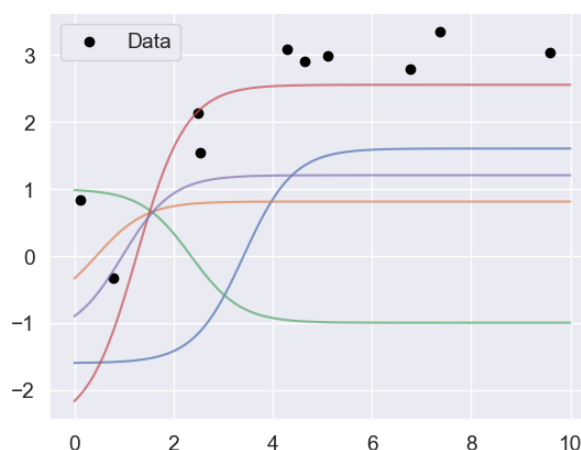


Figure 2.1: Functions $f(x)$ from 100 generated samples from prior $p(w)$ on top of scatter plot of given data

2.2 Task 2.2 | What prevents us from using the equations in section 3.3 in Murphy¹ to compute the exact posterior distribution of the parameters given the data analytically for the system in eq. (14) ?

This is because in section 3.3, the equations and derivations rely on linear Gaussian models, but in our case, we have a sigmoid function in equation (10), and since this one is not linear, it prevents us from using the equations from section 3.3.

SO, as a result, the likelihood is no longer Gaussian in w , and the posterior cannot be computed analytically. Because, in short, since the likelihood is not Gaussian in w , the posterior is not Gaussian and the integral for the posterior has no closed-form solution.

Since there is non-linearity in parameters, we cannot use the equations in section 3.3 to compute the exact posterior distribution of the parameters analytically.

2.3 Task 2.3 | Implement a python for function for evaluating the logarithm of the joint density in eq. (14). Report the numerical value of $\log p(y, w)$ for the dataset given above when $w_0 = w_1 = 0$.

We write the following code in Python, assuming we have already defined the data:

```
1 tau = 2
2 beta = 4
3
4 def sigma(x):
5     return np.tanh(x)
6
7 def f(x, w):
8     return w[1] * sigma(x + w[0])
9
10 def log_prior(w):
11     D = len(w)
12     return -0.5 * np.sum(w**2) / tau**2 - (D/2) * np.log(2*np.pi*tau**2)
13
14 def log_likelihood(x, y, w):
15     y_pred = f(x, w)
16     return -0.5 * beta * np.sum((y - y_pred)**2) + len(y)/2 * np.log(beta/(2*np.pi))
17
18 def log_joint(x, y, w):
19     return log_prior(w) + log_likelihood(x, y, w)
20
21 # evaluate at w = [0, 0]
22 w = np.array([0.0, 0.0])
23 value = log_joint(x, y, w)
24
25 print(value)
```

This is from considering a Bayesian regression model where the goal is to evaluate the joint density

$$\log p(\mathbf{y}, \mathbf{w}) = \log p(\mathbf{y} | \mathbf{w}) + \log p(\mathbf{w}) \quad (2.1)$$

Where we assume the following model, first the prior on the parameters:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}) \quad (2.2)$$

Then, again, we got the non-linear model $f(x, \mathbf{w}) = w_1 \sigma(x + w_0)$, and since the prior is a multivariate Gaussian we can write the following for the log prior:

$$\log p(\mathbf{w}) = -\frac{1}{2\tau^2} \mathbf{w}^T \mathbf{w} - \frac{D}{2} \log(2\pi\tau^2) \quad (2.3)$$

Furthermore, assuming independent Gaussian noise for each observation:

$$y_n \sim \mathcal{N}(f(x_n, \mathbf{w}), \beta^{-1}) \quad (2.4)$$

The log likelihood becomes:

$$\log p(\mathbf{y} | \mathbf{w}) = -\frac{1}{2}\beta \sum_{n=1}^N (y_n - f(x_n, \mathbf{w}))^2 + \frac{N}{2} \log\left(\frac{\beta}{2\pi}\right) \quad (2.5)$$

Finally, the log joint density is the sum of the log prior and log likelihood which is:

$$\log p(\mathbf{y}, \mathbf{w}) = \log p(\mathbf{y} | \mathbf{w}) + \log p(\mathbf{w}) \quad (2.6)$$

Then, after running the code, we find that the numerical value of the log joint density for the dataset when $w_0 = w_1 = 0$ is -130.95 .

Therefore, the numerical value of $\log p(y, w)$ is -130.95 .

2.4 Task 2.4 | Determine the expression for the gradient and Hessian of the logarithm of the joint density wrt. $\mathbf{w}$.

From 2.6 we see that the log joint density can be split into the log prior and the log likelihood, so we have to take the Hessians to both of those and sum. We start off with the prior, first becomes:

$$\nabla \log p(w) = -\frac{1}{\tau^2} w \quad (2.7)$$

Because differentiating $w^T w$ gives us $2w$, and then differentiating once more to get the hessian, which becomes:

$$\nabla^2 \log p(w) = -\frac{1}{\tau^2} I \quad (2.8)$$

We have to do the same for the log likelihood:

$$\nabla \log p(y|w) = \beta \sum_{n=1}^N (y_n - f(x_n, w)) \nabla f(x_n, w) \quad (2.9)$$

Then for the hessian:

$$\nabla^2 \log p(y|w) = \beta \sum_{n=1}^N [-\nabla f \nabla f^T + (y - f(x_n, w)) \nabla^2 f] \quad (2.10)$$

Therefore, we determine the gradient of the log joint density to be:

$$\nabla \log p(y, w) = \beta \sum_{n=1}^N (y_n - f(x_n, w)) \nabla f(x_n, w) - \frac{1}{\tau^2} w \quad (2.11)$$

And the hessian of the log joint density is determined to be:

$$\nabla^2 \log p(y, w) = \beta \sum_{n=1}^N [-\nabla f \nabla f^T + (y - f(x_n, w)) \nabla^2 f] - \frac{1}{\tau^2} I \quad (2.12)$$

We have now determined the expression for the gradient and hessian of the log joint density with respect to w .

2.5 Task 2.5 | Determine the Laplace approximation $q(w)$ of the posterior distribution such that $p(w|y) \approx q(w)$.

In order to do this, we have to find MAP first $\hat{w} = \arg \max_w \log p(y|w) + \log p(w)$ and we do this via Python.

```

1 def log_prior(w):
2     D = len(w)
3     return -0.5 * np.sum(w**2) / tau**2 - (D/2) * np.log(2*np.pi*tau**2)
4
5 def log_likelihood(x, y, w):
6     y_pred = f(x, w)
7     return -0.5 * beta * np.sum((y - y_pred)**2) + len(y)/2 * np.log(beta/(2*np.pi))
8
9 def log_joint(x, y, w):
10    return log_prior(w) + log_likelihood(x, y, w)
11
12 # minimize expects a FUNCTION, and minimizes, so we negate log_joint
13 result = minimize(
14     fun=lambda w: -log_joint(x, y, w), # pass a function, not a value
15     x0=np.array([0.0, 0.0]), # initial guess for w
16 )
17
18 w_map = result.x
19 print("MAP estimate:", w_map)
20 print("Optimization success:", result.success)

```

We get a MAP estimate of $[-0.31, 2.62]$. We plug this into the hessian of the joint which we found above in equation 2.12. Below is a code for the hessian (which I got a bit of help from):

```

1 def sigma_prime(x):
2     return 1 - np.tanh(x)**2 # derivative of tanh
3
4 def sigma_double_prime(x):
5     return -2 * np.tanh(x) * sigma_prime(x) # second derivative of tanh
6
7 def grad_f(x, w):
8     """Gradient of f w.r.t. w: returns shape (2,)"""
9     s_prime = sigma_prime(x + w[0])
10    df_dw0 = w[1] * s_prime # f / w0
11    df_dw1 = sigma(x + w[0]) # f / w1
12    return np.array([df_dw0, df_dw1])
13
14 def hess_f(x, w):
15    """Hessian of f w.r.t. w: returns shape (2, 2)"""
16    s_prime = sigma_prime(x + w[0])
17    s_double = sigma_double_prime(x + w[0])
18    # second derivatives of f:
19    d2f_dw0w0 = w[1] * s_double # 2f / w0^2

```

```

20     d2f_dw0w1 = s_prime #  $\partial^2 f / \partial w_0 \partial w_1$ 
21     d2f_dw1w1 = 0.0 #  $\partial^2 f / \partial w_1^2 = 0$  (f is linear in  $w_1$ )
22     return np.array([[d2f_dw0w0, d2f_dw0w1],
23                      [d2f_dw0w1, d2f_dw1w1]])
24
25 def grad_log_joint(x, y, w):
26     """Gradient of log p(y, w) w.r.t. w"""
27     grad = np.zeros(2)
28     for xn, yn in zip(x, y):
29         gf = grad_f(xn, w)
30         grad += beta * (yn - f(xn, w)) * gf
31     grad -= w / tau**2 # gradient of log prior
32     return grad
33
34 def hessian_log_joint(x, y, w):
35     """Hessian of log p(y, w) w.r.t. w"""
36     H = np.zeros((2, 2))
37     for xn, yn in zip(x, y):
38         gf = grad_f(xn, w).reshape(-1, 1) # shape (2,1)
39         hf = hess_f(xn, w) # shape (2,2)
40         H += beta * (-gf @ gf.T + (yn - f(xn, w)) * hf)
41     H -= np.eye(2) / tau**2 # Hessian of log prior
42     return H
43
44 w_map = result.x
45 H = hessian_log_joint(x, y, w_map)
46 S = -np.linalg.inv(H) # posterior covariance
47
48 print("Covariance S:\n", S)

```

We recall that the final step is to approximate $p(w|y) \approx q(w) = \mathcal{N}(w|\hat{w}, S)$ where $S = (-H)^{-1}$ so we get:

$$q(w) = \mathcal{N}\left(w \mid \begin{bmatrix} -0.31 \\ 2.62 \end{bmatrix}, \begin{bmatrix} 0.0378 & -0.0034 \\ -0.0034 & 0.031 \end{bmatrix}\right) \quad (2.13)$$

Hence, we have found the laplace approximation $q(w)$ along with the posterior distribution such that $p(w|y) = q(w)$.

2.6 Task 2.6 | Create a 2D contour plot of posterior distribution of the parameters. Plot the contours of the Laplace approximation on top of the plot to validate your results from the previous tasks.

We write the following code for plotting:

```

1     from scipy.stats import multivariate_normal as mvn
2
3     w0_vals = np.linspace(-2, 2, 100)
4     w1_vals = np.linspace(0, 5, 100)
5     W0, W1 = np.meshgrid(w0_vals, w1_vals)
6
7     log_joint_grid = np.zeros_like(W0)
8     for i in range(W0.shape[0]):
9         for j in range(W0.shape[1]):
10             w_ij = np.array([W0[i, j], W1[i, j]])
11             log_joint_grid[i, j] = log_joint(x, y, w_ij)
12

```

```

13 laplace_grid = np.zeros_like(W0)
14 for i in range(W0.shape[0]):
15     for j in range(W0.shape[1]):
16         w_ij = np.array([W0[i, j], W1[i, j]])
17         laplace_grid[i, j] = mvn.logpdf(w_ij, mean=w_map, cov=S)
18
19 fig, ax = plt.subplots(figsize=(8, 6))
20 # true posterior (unnormalized) -blue
21 ax.contour(w0_vals, w1_vals, np.exp(log_joint_grid), levels=15, colors='blue', alpha=0.7)
22 # Laplace approximation q(w) -red
23 ax.contour(w0_vals, w1_vals, np.exp(laplace_grid), levels=15, colors='red', alpha=0.7,
24           linestyle='dashed')
25 # mark the MAP
26 ax.plot(w_map[0], w_map[1], 'ko', markersize=8, label='$\\hat{w}_{MAP}$')
27 ax.set(xlabel='$w_0$', ylabel='$w_1$', title='Posterior $p(w|y)$ vs Laplace approximation $q(w)$')
28 plt.show()

```

Which plots:

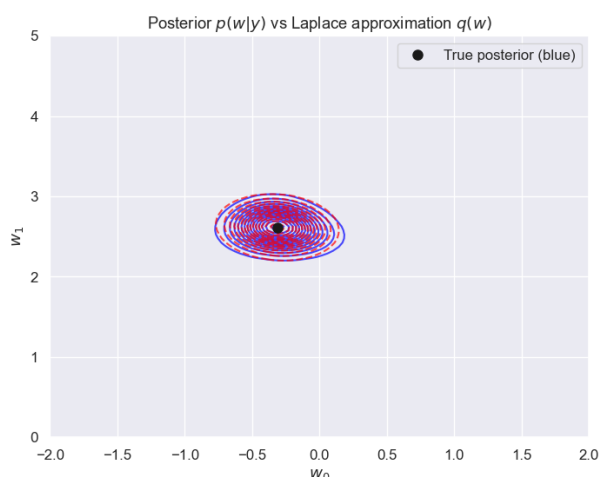


Figure 2.2: Posterior $p(w|y)$ versus the laplace approximation of, with true posterior being blue, and laplace approximation is red $q(w)$

We have now plotted the contour plot.

2.7 Task 2.7 | Generate and plot $S = 100$ samples from the approximate posterior distribution. Plot the corresponding function $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.

Now instead of sampling from the prior as we did in Task 2.1, we sample from the Laplace posterior:

```

1 x_plot = np.linspace(0, 10, 100)
2
3 fig, ax = plt.subplots(figsize=(8, 6))
4
5 # Sample from approximate posterior q(w) = N(w_map, S)
6 for _ in range(100):

```

```

7     w_sample = np.random.multivariate_normal(mean=w_map, cov=S)
8     ax.plot(x_plot, f(x_plot, w_sample), alpha=0.1, color='blue')
9
10    ax.scatter(x, y, color='black', zorder=5, label='Data')
11    ax.set(xlabel='$x$', ylabel='$f(x)$', title='100 samples from approximate posterior $q(w)$')
12    ax.legend()
13    plt.show()

```

We get:

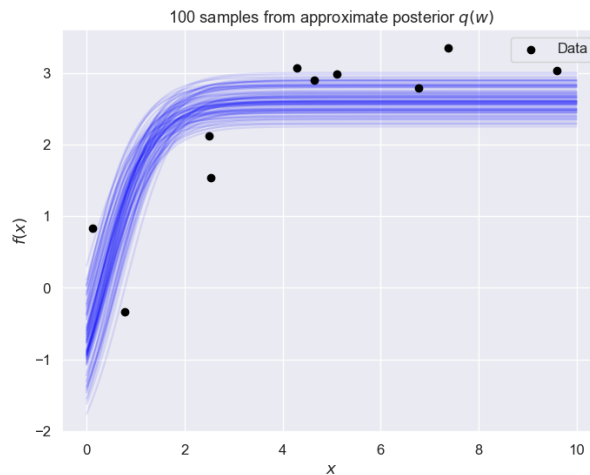


Figure 2.3: Sampling from approximate posterior distribution versus data points

We have now generated 100 samples from the approximate posterior distribution.

2.8 Task 2.8 | Use the Laplace approximation to compute the (approximate) posterior probability for the event $f(8) > 3$.

First we evaluate via $f(8, w_{\text{map}})$ to find the mean, and then the variance is $\text{Var}[f(8)] \approx \nabla f(8)^T S \nabla f(8)$, then we can compute the probability:

$$p(f(8) > 3) = 1 - p(f(8) \leq 3) = 1 - \Phi((3 - \mu)/\sigma) \quad (2.14)$$

We calculate via Python code:

```

1  # 1. mean of f(8) under Laplace approximation
2  mu_f8 = f(8, w_map)
3
4  # 2. variance via delta method: grad_f(8)^T @ S @ grad_f(8)
5  g = grad_f(8, w_map)
6  var_f8 = g @ S @ g
7  std_f8 = np.sqrt(var_f8)
8
9  prob = 1 - norm.cdf(3, loc=mu_f8, scale=std_f8)
10
11 prob

```

We find that $p(f(8) > 3) = 0.014$.

2.9 Task 2.9 | Determine the mean and a 95% interval for the (approximate) distribution $p(y^*|y, x^*)$ for $x^* = 3.75$.

First off, we recall that:

$$p(y^*|y, w) = \mathcal{N}(y_n|f(x), \beta^{-1}) \quad (2.15)$$

Furthermore, we have the distribution over w being the $q(w)$, so we can marginalize out w :

$$p(y^*|y, x^*) = \int p(y^*|w, x^*)q(w)dw \quad (2.16)$$

Then we somehow apply Murphy 3.3 and end up with $p(y^*|y, x^*) = \mathcal{N}(y^*|\mu_{y^*}, \sigma_{y^*}^2)$ which has:

$$\mu_{y^*} = f(x^*, \hat{w}) \quad (2.17)$$

$$\sigma_{y^*}^2 = \nabla f(x^*)^T S \nabla f(x^*) + \beta^{-1} \quad (2.18)$$

With $\nabla f(x^*)^T S \nabla f(x^*)$ being uncertainty in w and the other β^{-1} being noise from observations. Now it is all up to Python:

```
1 x_star = 3.75
2
3 # mean
4 mu_ystar = f(x_star, w_map)
5
6 # variance = epistemic + aleatoric
7 g = grad_f(x_star, w_map)
8 var_ystar = g @ S @ g + 1/beta
9
10 # 95% interval
11 std_ystar = np.sqrt(var_ystar)
12 lower = mu_ystar - 1.96 * std_ystar
13 upper = mu_ystar + 1.96 * std_ystar
14
15 print(f"Mean: {mu_ystar:.4f}")
16 print(f"95% interval: [{lower:.4f}, {upper:.4f}"])
```

We find a mean of 2.6 and interval of [1.57, 3.65].

We have hereby finished this exercise.